ELO320 - Estructuras de Datos y Algoritmos
QuickSort
Divide & Conquer Parte II

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

2do Semestre 2022

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

QuickSort

Es un algoritmo de ordenamiento **no estable** de complejidad promedio $\mathcal{O}(n \log n)$, sin embargo puede en sus peores casos llegar a $\mathcal{O}(n^2)$. Al igual que MergeSort utiliza la tecnica de Divide & Conquer.

QuickSort es unos de los algoritmos predilectos de ordenamiento actualmente:

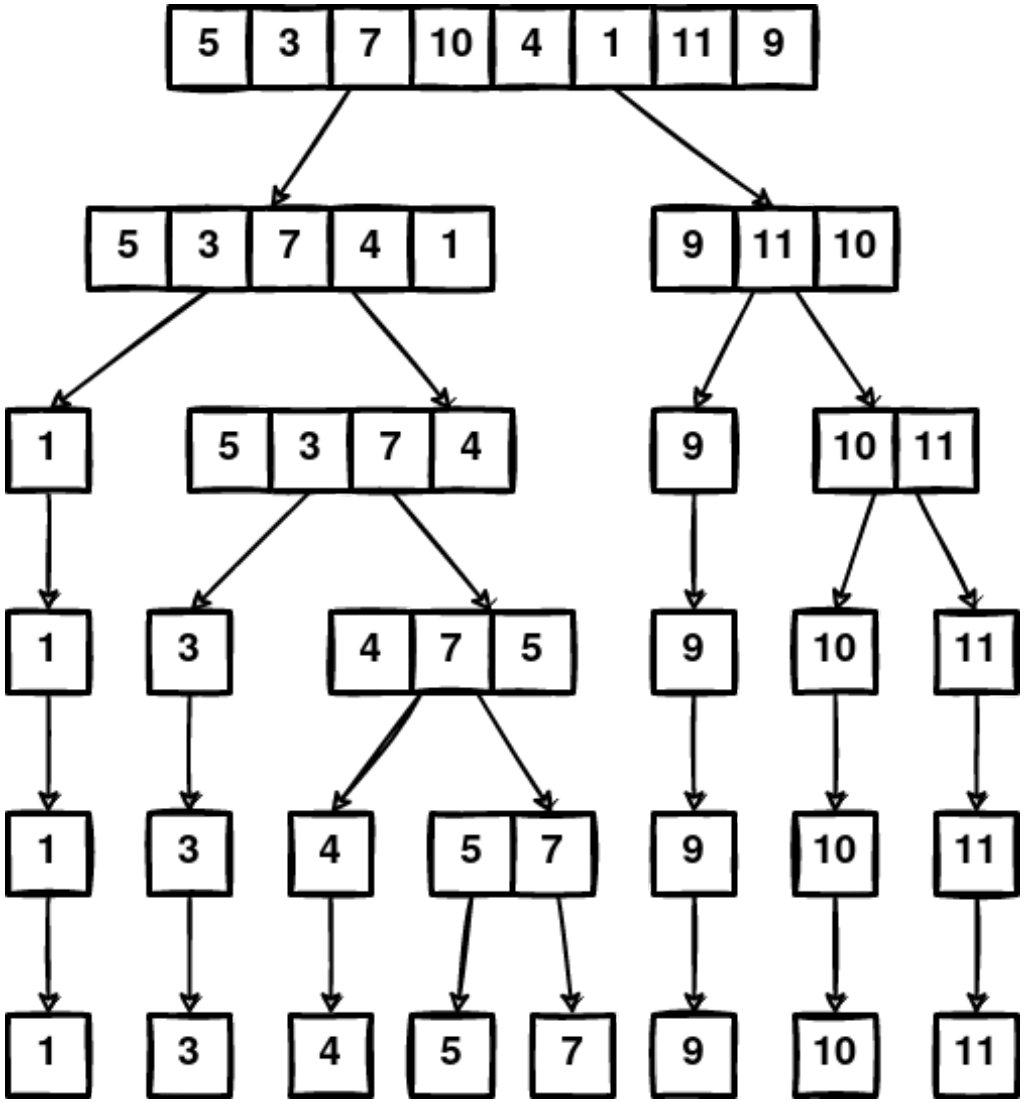
- Usa recursividad.
- No usa estructuras de datos anexas.
- Existen técnicas para evitar casos de complejidad $\mathcal{O}(n^2)$.
- Naturalmente modelado para disminuir complejidad.

QuickSort

Quicksort tiene **tres pasos**:

1. Dividir: Separa el arreglo en otros dos, donde los miembros de uno son siempre menor que el otro.
2. Conquistar: Ordenar los sub-arreglos, generados.
3. Combinar: Como el resultado final siempre estará en orden, combinar es una tarea trivial.

QuickSort: Idea General



Nota: esto es sólo la idea general y no como exactamente funciona.

Esquemas de Partición

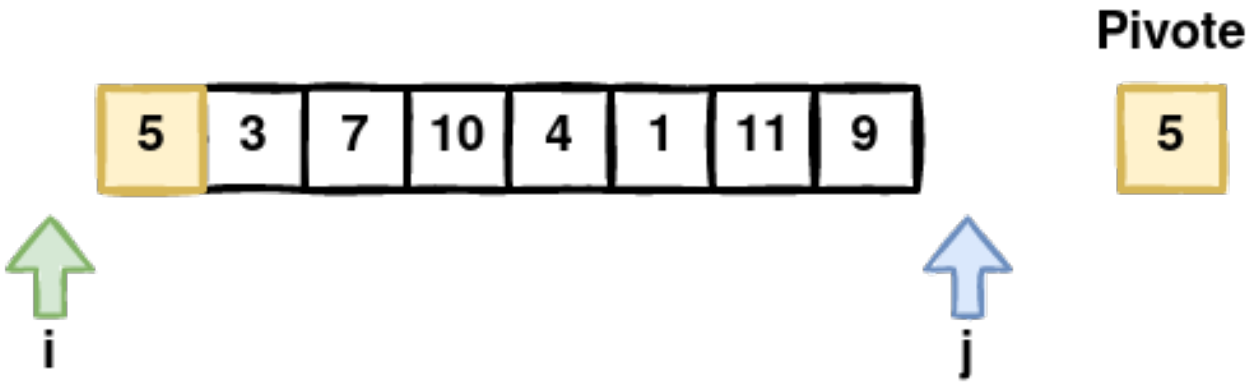
QuickSort se basa en un esquema de particiones que puede ser implementado de variadas formas. Estos esquemas dependen de un valor pivote, que permite ordenar los sub-arreglos.

Generalmente, se comparan dos de las formas mas utilizadas:

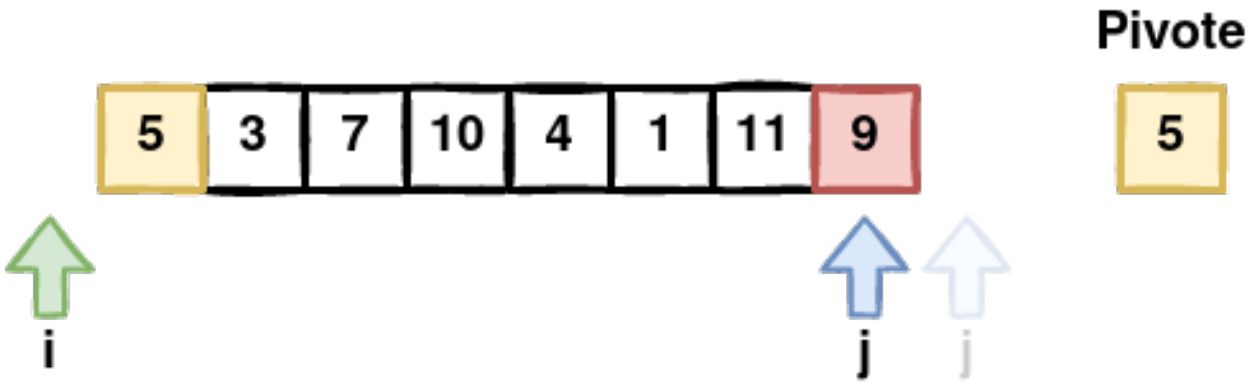
- Forma Lomuto: Pivote al final. Facil implementación. Redundante.
- Forma Hoare: Forma original. Pivote al inicio. Sofisticado. Muy eficiente.

En este curso veremos la forma original: El **Esquema de Hoare**.

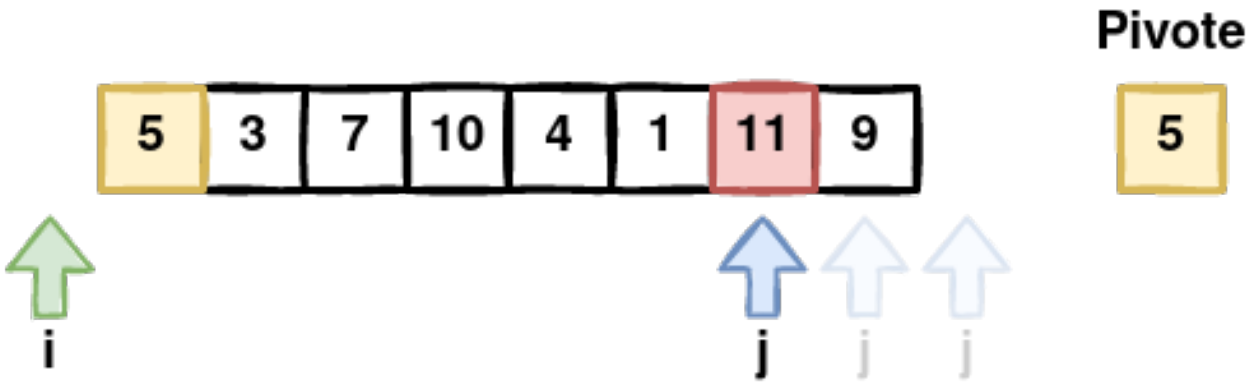
Partition: Hoare's Scheme



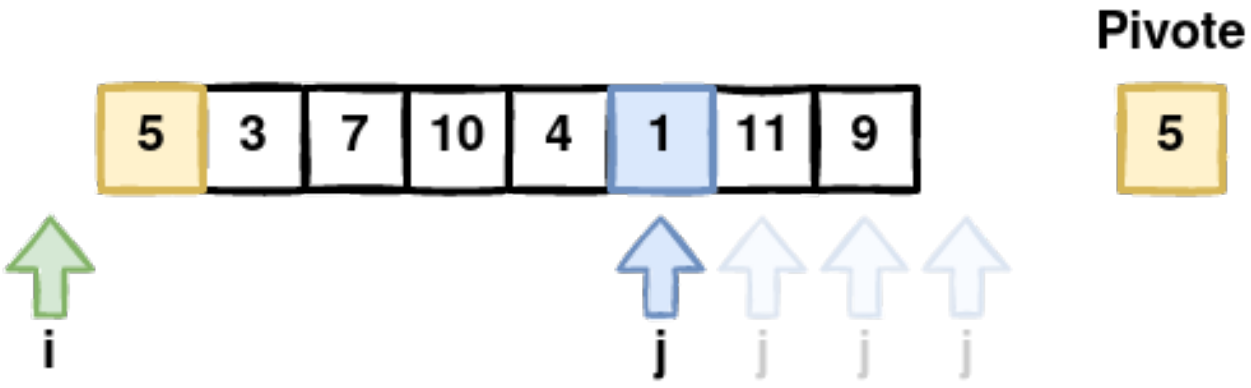
Partition: Hoare's Scheme



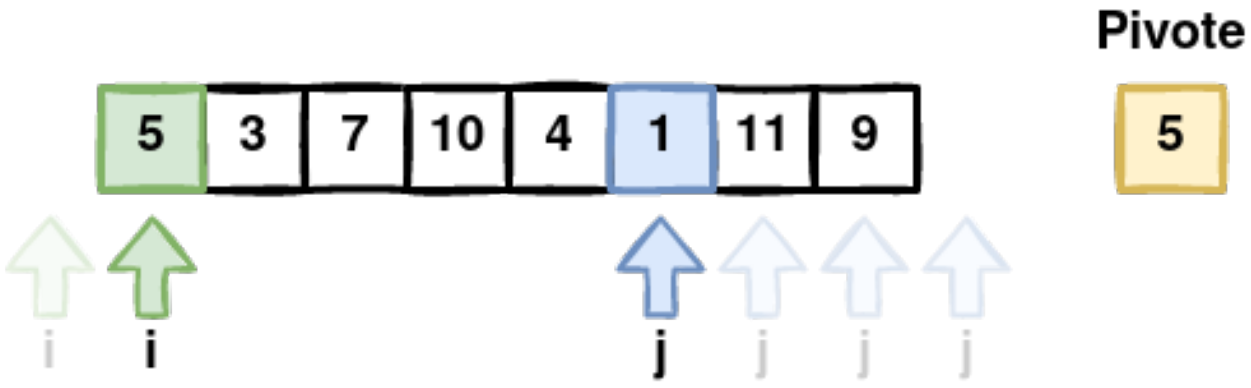
Partition: Hoare's Scheme



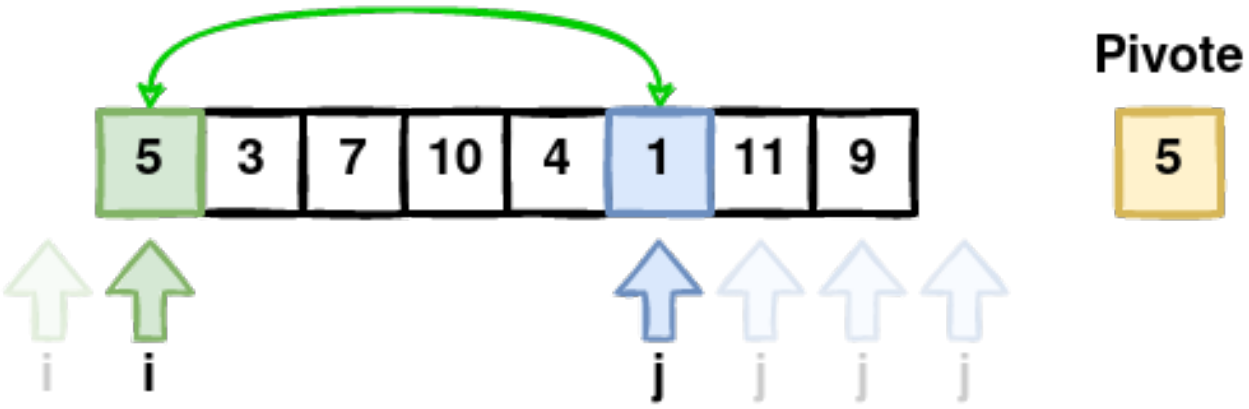
Partition: Hoare's Scheme



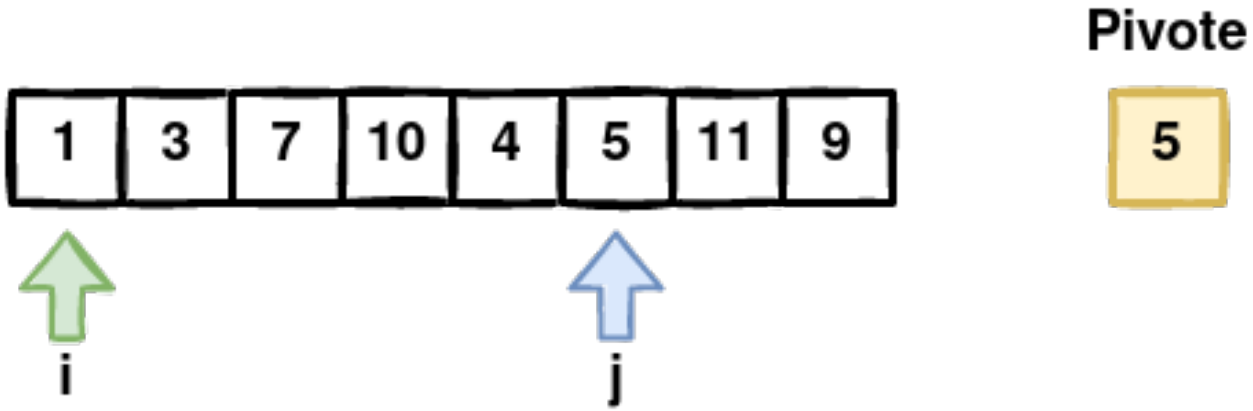
Partition: Hoare's Scheme



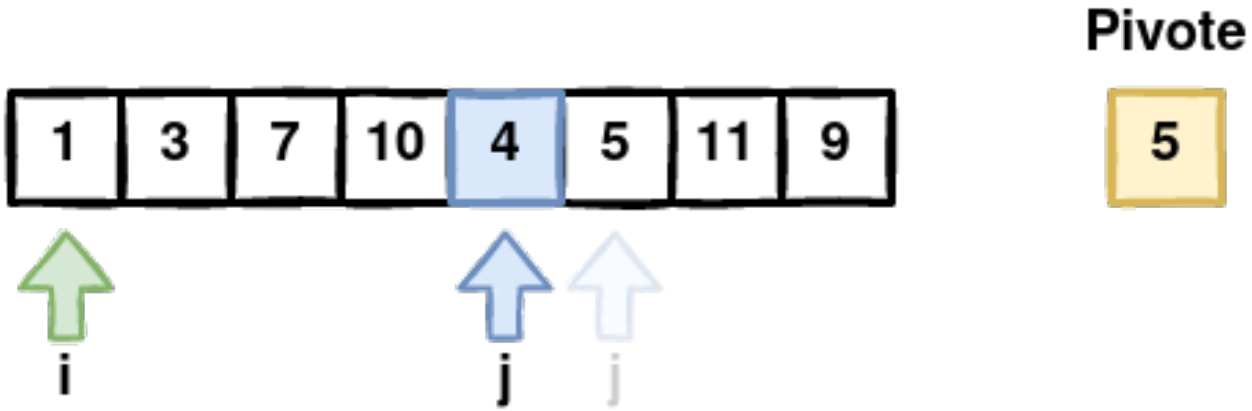
Partition: Hoare's Scheme



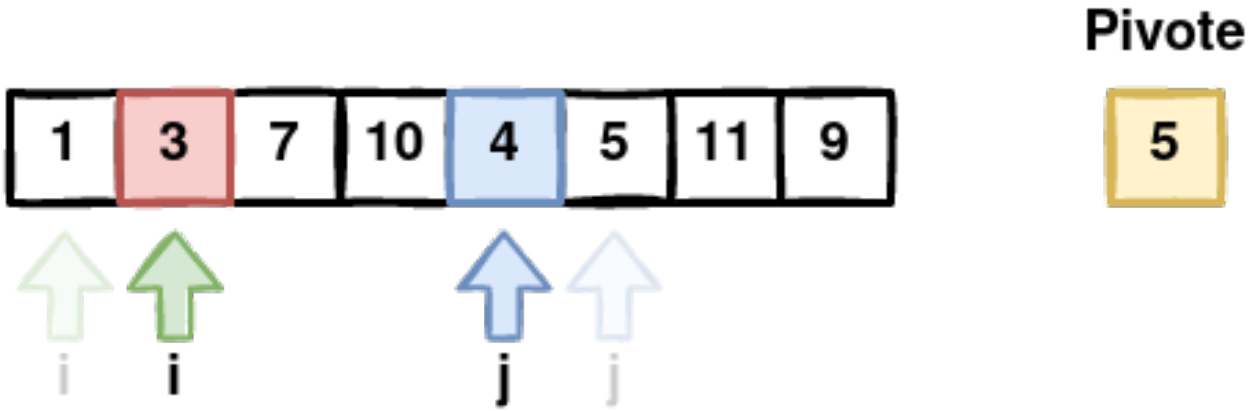
Partition: Hoare's Scheme



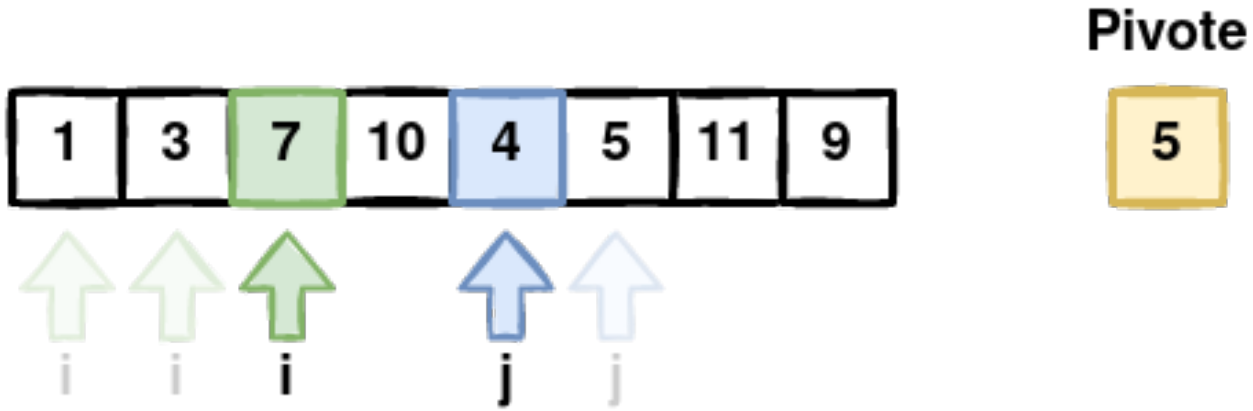
Partition: Hoare's Scheme



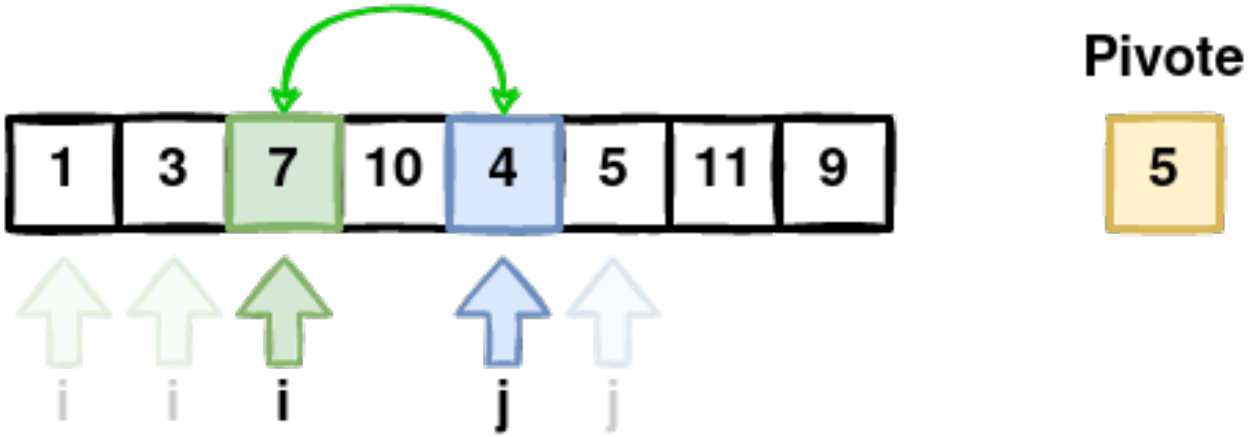
Partition: Hoare's Scheme



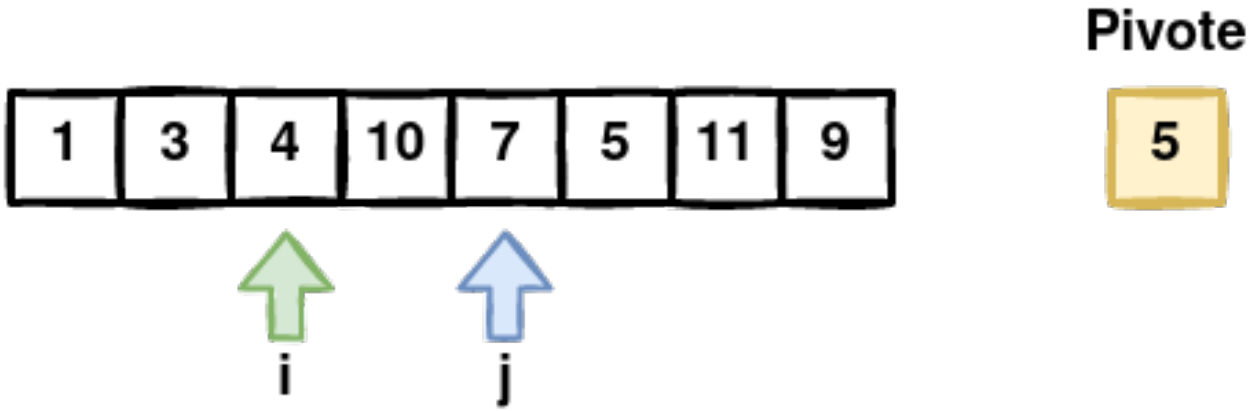
Partition: Hoare's Scheme



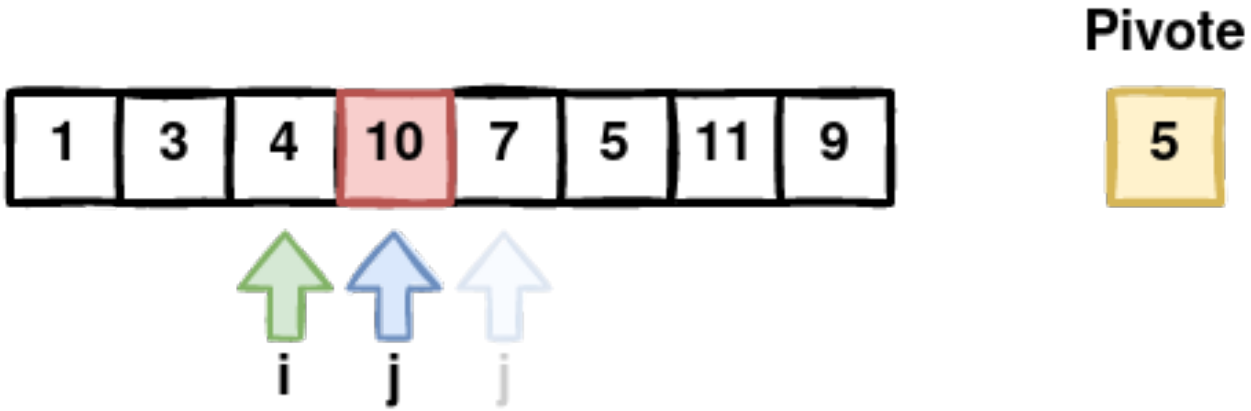
Partition: Hoare's Scheme



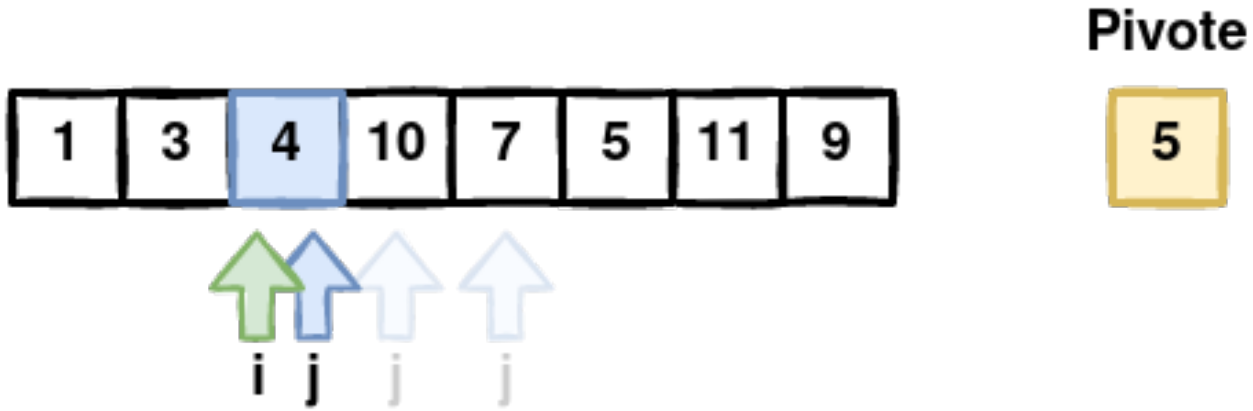
Partition: Hoare's Scheme



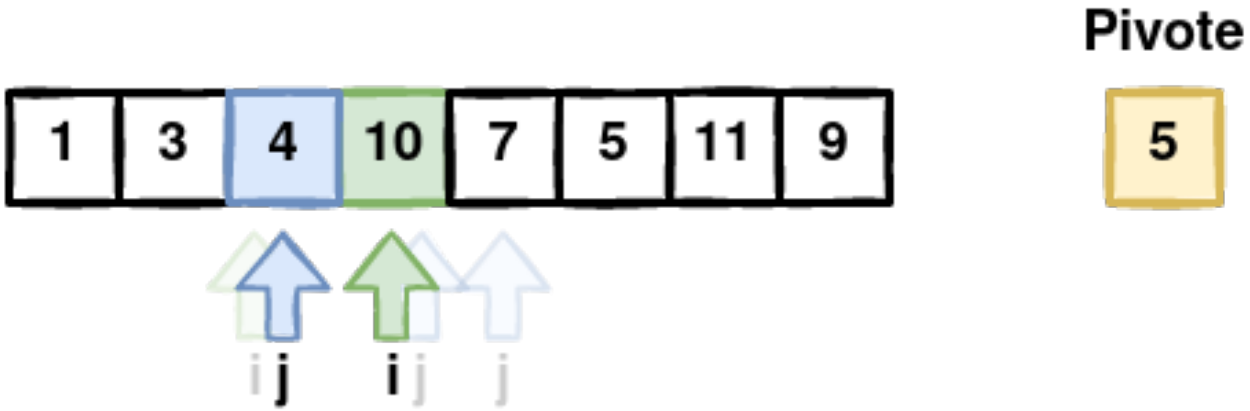
Partition: Hoare's Scheme



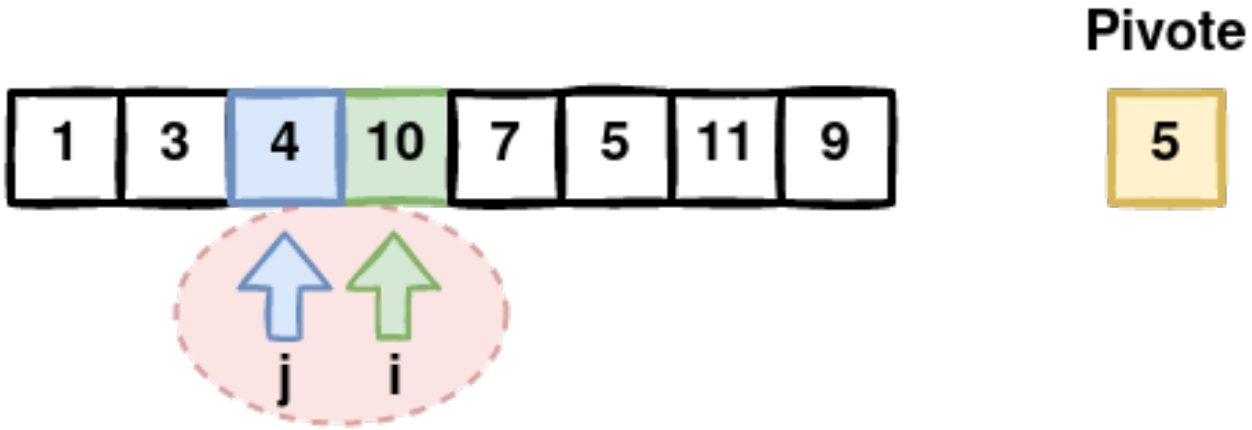
Partition: Hoare's Scheme



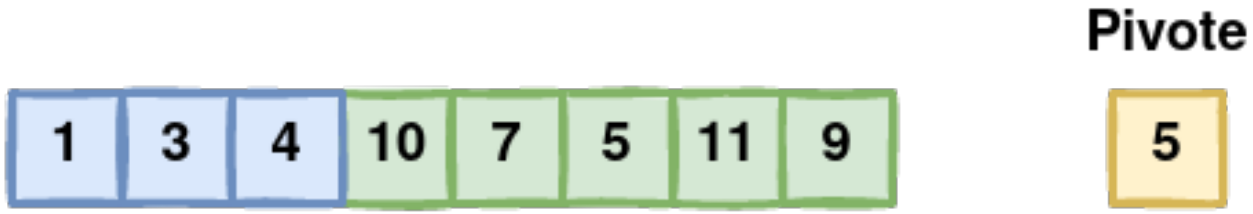
Partition: Hoare's Scheme



Partition: Hoare's Scheme



Partition: Hoare's Scheme



Partition: Hoare's Scheme

Partition: Hoare's Scheme

Partition

```
int partition(int arr[], int inicio, int fin){
    int pivote = arr[inicio];
    int i = inicio - 1, j = fin + 1, temp;

    while (i<j){
        do{
            i++;
        }while (arr[i] < pivote);

        do{
            j--;
        }while (arr[j] > pivote);

        if(j<=i)
                break;

        temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    return j;
}
```

QuickSort

Quicksort en sí es un procedimiento sencillo.

QuickSort

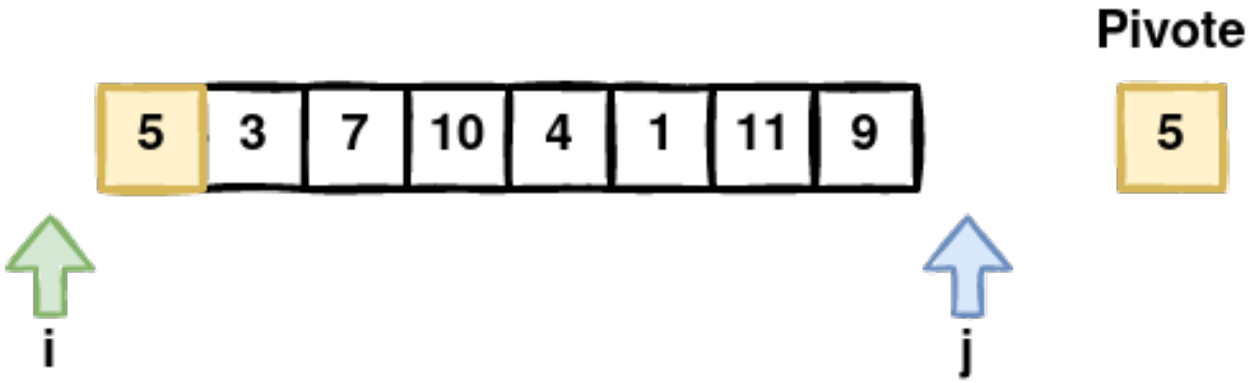
```
void quickSort(int arr[], int inicio, int fin){
    if (inicio < fin) {

        int corte = partition(arr, inicio, fin);

        quickSort(arr, inicio, corte);
        quickSort(arr, corte + 1, fin);
    }
}
```

QuickSort

Apliquemos QuickSort sobre el siguiente arreglo:



QuickSort Aleatorizado

¿Cuál es el peor escenario de QuickSort?
Es cuando siempre nos queda una partición de 1 y otra de n-1.

- Siempre tendremos que practicamente recorrer completamente el arreglo.
- La recursión no será de largo $\log n$ sino n.
- Por lo tanto la complejidad en el peor escenario es $\mathcal{O}(n^2)$.
- El caso de la mala suerte (literal).

Para evitar eso, la elección del pivote se hace aleatoria, de esa forma el tiempo **esperado** (concepto estadístico) del peor escenario es $\mathcal{O}(n \log n)$. (Nota: el peor caso sigue siendo $\mathcal{O}(n^2)$).

QuickSort Aleatorizado

QuickSort Aleatorizado

```
void quickSort(int arr[], int inicio, int fin){
    if (inicio < fin) {

        int corte = random_partition(arr, inicio, fin);

        quickSort(arr, inicio, corte);
        quickSort(arr, corte + 1, fin);
    }
}
```

QuickSort Aleatorizado

Random Partition

```
int random_partition(int arr[], int inicio, int fin){
    int index = rand() % (fin + 1);
    int pivote = arr[index];
    int i = inicio - 1, j = fin + 1, temp;

    while (i<j){
        do{
            i++;
        }while (arr[i] < pivote);

        do{
            j--;
        }while (arr[j] > pivote);

        if(j<=i)
                break;

        temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    return j;
}
```

Ejercicios

- Programa y dibuje la operación `partition` sobre el arreglo $A = \{13; 19; 9; 5; 12; 8; 7; 4; 21; 2; 6; 11\}$.
- ¿Qué valor retorna `partition` cuando todos los elementos del arreglo son iguales?
- ¿Cuál es el tiempo de ejecución de `QuickSort` cuando todos los elementos del arreglo son iguales?